



Trabajo de Diploma

Informe Patrones

Integrantes: Cores Juan Ignacio, De Mieri Facundo y Ricci Agustin.

Año: 2026.

Turno: Noche.

Comisión: T1-23-32.

Materia: Trabajo de Diploma.

Docente: Leonel Jimenez Gamboa.

Patrones: Builder, Bridge y Chain of Responsibility

Builder

Clasificación

Builder es un patrón creacional del catálogo GoF. Este tipo de patrones resuelven cómo se crean los objetos

Descripción (que hace)

Separa la construcción de un objeto complejo de su representación. Así, el mismo proceso de construcción puede generar distintas representaciones.

En lugar de crear el objeto de una sola vez con un constructor que recibe muchos parámetros, el objeto se arma paso a paso. Un objeto constructor se encarga de cada parte. El producto se entrega recién cuando está completo y es válido.

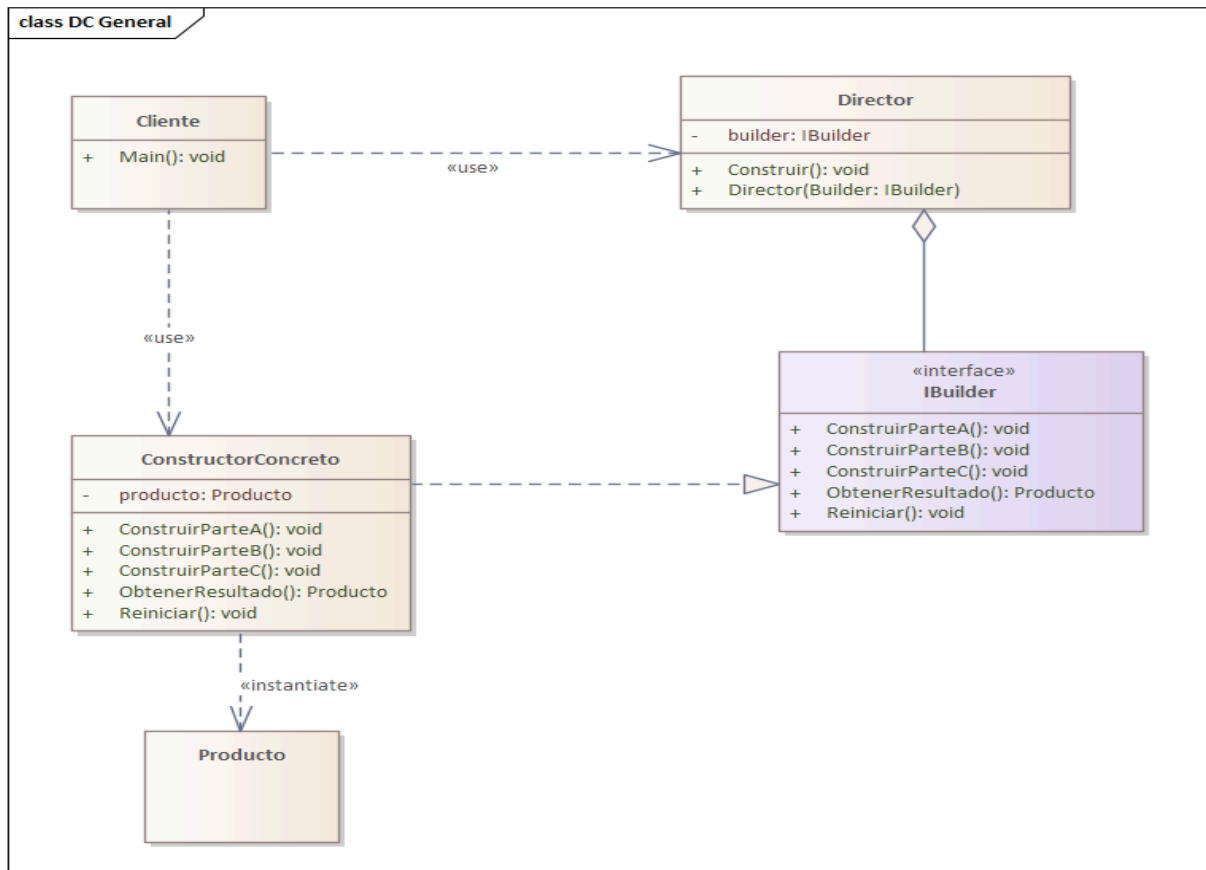
Problema que resuelve

- **Constructor telescópico:** Clases con constructores de muchos parámetros, muchos opcionales. Esto lleva a múltiples sobrecargas difíciles de leer y propensas a errores.
- **Objetos a medio construir:** Si se usan setters sueltos, el objeto puede quedar en un estado invalido o incompleto.
- **Lógica de armado duplicada:** cuando el mismo proceso se repite para crear variantes distintas de un producto.

Cuando se aplica

- Cuando el algoritmo para crear un objeto complejo debe ser *independiente de las partes* que lo componen y de cómo se ensamblan.
- Cuando el proceso de construcción debe permitir *distintas representaciones* del objeto construido.
- Cuando un objeto tiene *muchos atributos opcionales* o configuraciones posibles.
- Cuando la construcción requiere *pasos en un orden determinado o validaciones* antes de entregar el objeto.
- Cuando se quiere que el producto final sea *inmutable* una vez creado.

Diagrama de Clases General



Participantes

Participante	Responsabilidad
Producto	Objeto complejo que se construye. Tiene muchas partes.
Builder (interfaz o clase abstracta)	Declara los métodos para construir cada parte del producto, por ejemplo ConstruirParteA(), ConstruirParteB() y ObtenerResultado().
Constructor Concreto	Implementa el Builder. Arma una representación específica del producto, guarda el estado parcial y devuelve el producto terminado.
Director	Define el orden en que se llaman los pasos de construcción. Encapsula “recetas” de construcción reutilizables.
Cliente	Elige el Constructor Concreto, se lo pasa al Director (o lo usa directamente) y obtiene el producto.

Ventajas

- Permite construir objetos paso a paso, diferir pasos o ejecutarlos según condiciones.
- Reutiliza el mismo código de construcción para distintas representaciones.
- Principio de Responsabilidad Única: aísla la lógica compleja de construcción de la lógica de negocio del producto.
- Código cliente más legible que los constructores con muchos parámetros.

Desventajas

- Aumenta la cantidad de clases: se necesita un builder por cada tipo de producto.
- Para objetos simples agrega complejidad innecesaria.
- El builder suele duplicar los atributos del producto.

Ejemplos de Aplicación

Ejemplo 1 - Zapatilla personalizada (se programa)

Consigna

Una marca de calzado vende zapatillas personalizables en su local. Cada zapatilla tiene modelo, color, tipo de suela y talla, y su precio depende de lo elegido. El vendedor puede:

- ofrecer un modelo prearmado (Running o Urbana), donde el cliente solo indica su talla, o;
- armar una zapatilla a medida, eligiendo cada parte desde un menú por teclado.

Al final, el sistema muestra la ficha de la zapatilla con su precio. Se valida que el talla esté entre 35 y 46 y que las opciones del menú sean válidas.

Justificación

La zapatilla se arma por partes y admite muchas combinaciones. Con Builder:

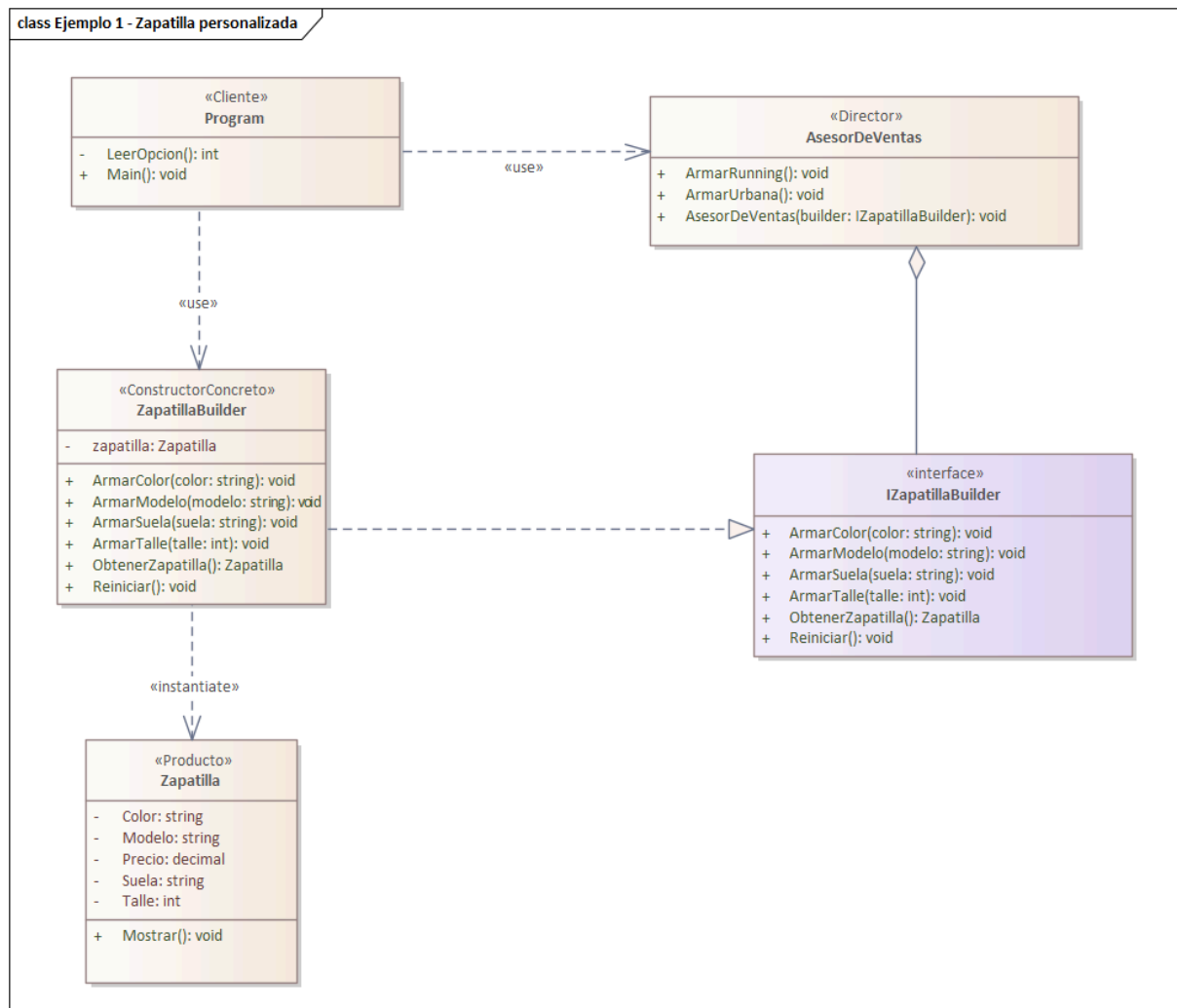
- las versiones pre armadas son recetas del Director (AsesorDeVentas), que llama siempre a los mismos pasos;
- el armado a medida usa el mismo builder, paso a paso, según lo que elige el usuario.

Así no hace falta un constructor con muchos parámetros, y agregar un nuevo modelo prearmado no obliga a tocar la clase Zapatilla.

Participantes

- **Producto:** Zapatilla.
- **Builder:** IZapatillaBuilder.
- **Constructor Concreto:** ZapatillaBuilder.
- **Director:** AsesorDeVentas.
- **Cliente:** Program.

Ejemplo 2 - Reportes en distintos formatos (se programa)



Consigna

Un comercio quiere generar un reporte de ventas de un período. El usuario carga por teclado el título, el período y las ventas (producto, cantidad y precio unitario), y luego elige el formato de salida: **texto plano o HTML**. El reporte siempre tiene la misma estructura: **encabezado, detalle de ventas, totales y pie**. El sistema muestra en pantalla el reporte generado en el formato elegido.

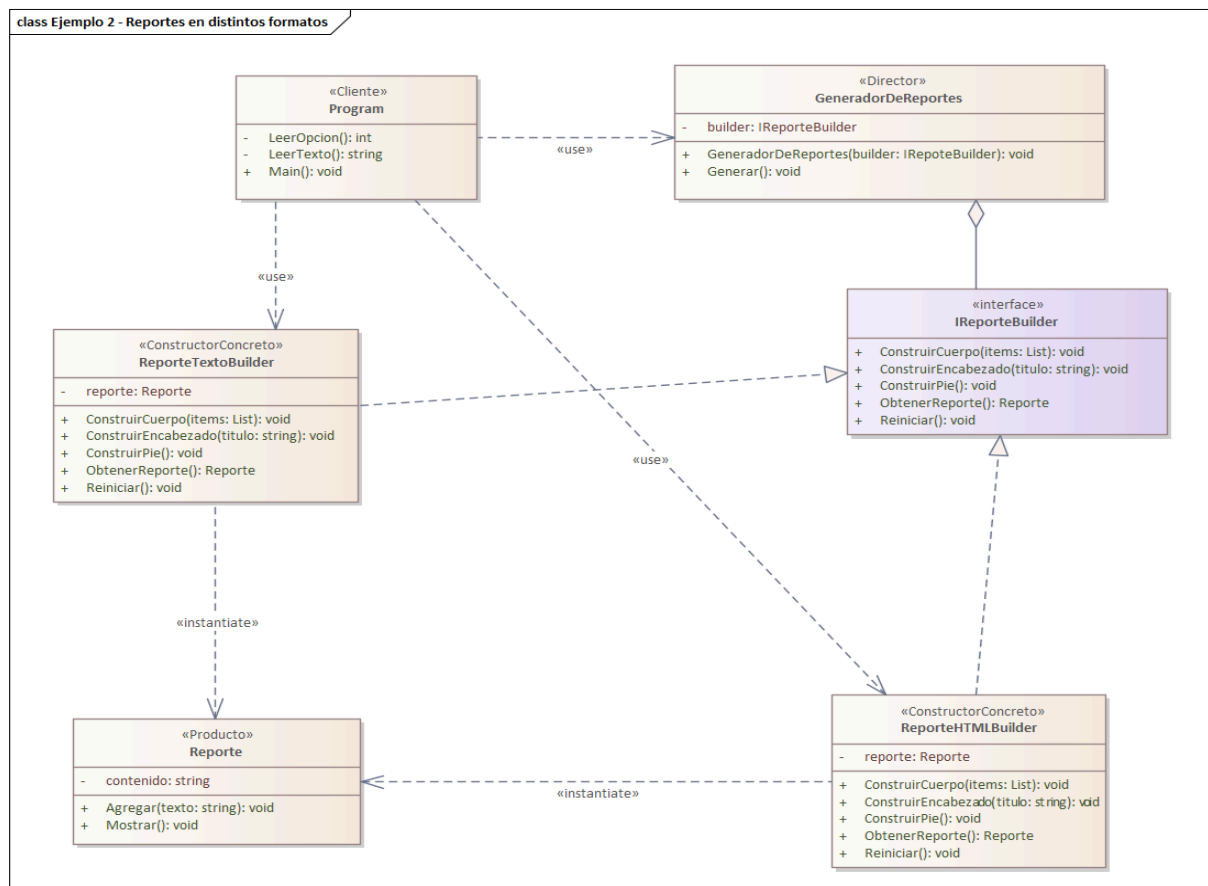
Este ejemplo muestra la idea central del Builder, ya que el mismo proceso de construcción genera distintas representaciones.

- El **Director** (GeneradorDeReportes) define siempre los mismos pasos, en el mismo orden.
- Cada **Constructor Concreto** sabe representar esos pasos en su formato.

Para agregar un nuevo formato, por ejemplo Markdown, solo se crea un nuevo builder, sin tocar el Director ni los demás builders. Esto cumple el **principio Abierto/Cerrado**.

Participantes

- **Producto:** Reporte.
- **Builder:** IReporteBuilder.
- **Constructores Concretos:** ReporteTextoBuilder, ReporteHtmlBuilder.
- **Director:** GeneradorDeReportes.
- **Cliente:** Program.
- **Clase auxiliar:** Venta.



Ejemplo 3 - Setup de un auto F1 (no se programa)

Consigna

Un equipo de Fórmula 1 prepara el auto antes de cada carrera. El procedimiento es siempre el mismo:

1. ajustar los alerones;
2. ajustar la suspensión;
3. elegir los neumáticos.

Pero cada circuito pide una configuración distinta:

- **Mónaco**, lento y lleno de curvas, necesita alta carga aerodinámica, suspensión blanda y neumáticos blandos;
- **Monza**, muy rápido y con largas rectas, necesita baja carga, suspensión dura y neumáticos duros.

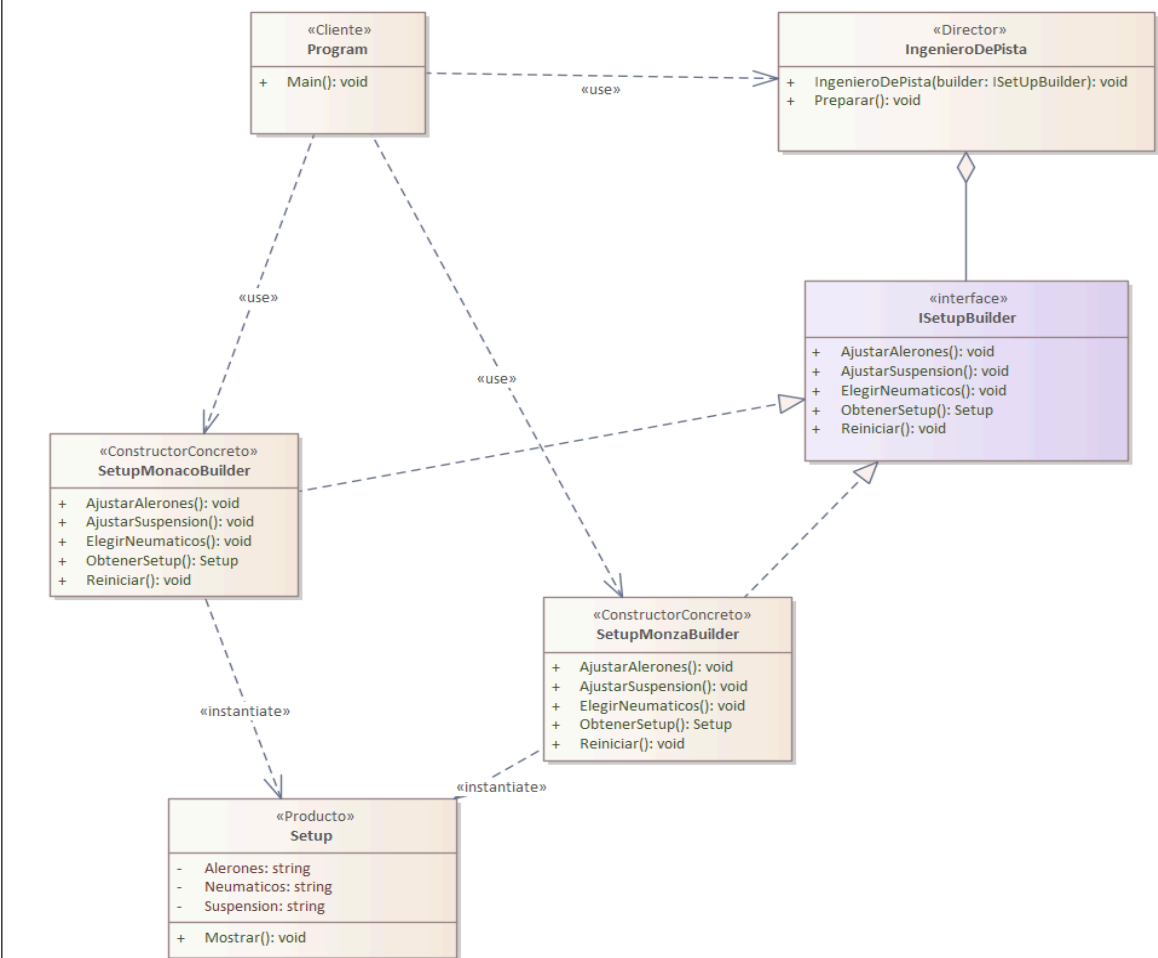
Justificación

Es el caso típico de mismo proceso, distinta representación. El Director (IngenieroDePista) siempre ejecuta los mismos tres pasos en el mismo orden, sin saber qué circuito se está preparando. Cada Constructor Concreto (SetupMonacoBuilder, SetupMonzaBuilder) define cómo se resuelve cada paso para su circuito. Para preparar el auto para un circuito nuevo, por ejemplo Spa, alcanza con agregar un nuevo builder, sin modificar al ingeniero ni al resto de las clases.

Participantes

- **Producto:** Setup.
- **Builder:** ISetupBuilder.
- **Constructores Concretos:** SetupMonacoBuilder, SetupMonzaBuilder.
- **Director:** IngenieroDePista.
- **Cliente:** Program.

class Ejemplo 3 - Setup de auto de F1



Bridge

Clasificación

Bridge es un patrón estructural del catálogo GoF. Este tipo de patrones resuelven cómo se combinan las clases y los objetos para formar estructuras más grandes y flexibles.

Descripción (qué hace)

Separa una abstracción de su implementación, de modo que ambas puedan variar de forma independiente. Para lograrlo, reemplaza una única jerarquía de herencia por dos jerarquías separadas que se unen por composición.

La abstracción define "qué" se hace y guarda una referencia a una interfaz, el implementador, que define "cómo" se hace. Cuando recibe una solicitud, la abstracción delega el trabajo de bajo nivel en ese implementador. Esa referencia es el "puente". Como el implementador se recibe por constructor, se puede combinar cualquier abstracción con cualquier implementación, e incluso cambiarla en tiempo de ejecución.

Problema que resuelve

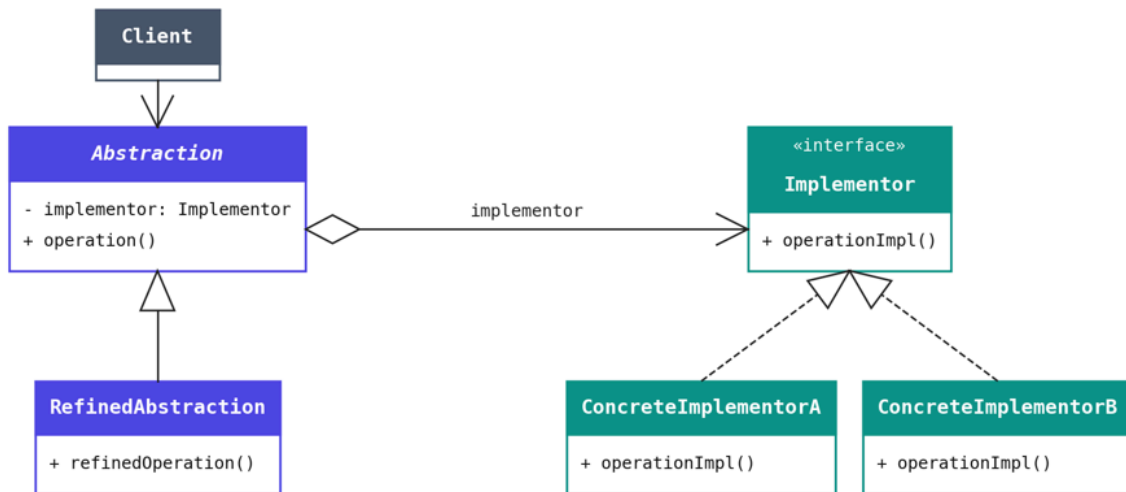
- **Explosión de clases:** cuando algo varía en dos dimensiones, la herencia obliga a crear una clase por cada combinación. Con M abstracciones y N implementaciones se necesitan $M \times N$ clases. Con Bridge se necesitan $M + N$.
- **Abstracción acoplada a su implementación:** si la implementación está fija dentro de la jerarquía, un cambio en un lado obliga a modificar el otro.
- **Implementación fija en compilación:** con herencia, la implementación se elige al declarar la clase y no se puede cambiar mientras el programa se ejecuta.
- **Código duplicado:** la misma lógica de bajo nivel se repite en cada combinación de abstracción e implementación.

Cuándo se aplica

- Cuando un concepto varía en dos dimensiones independientes y se quiere evitar una clase por cada combinación.
- Cuando se quiere evitar un enlace permanente entre una abstracción y su implementación.
- Cuando tanto la abstracción como la implementación deben poder extenderse con subclasses sin afectarse entre sí.
- Cuando la implementación debe poder cambiarse en tiempo de ejecución.

- Cuando se quiere ocultar los detalles de implementación al código cliente.

Diagrama de Clases General



Participantes

Participante	Responsabilidad
Abstracción (Abstraction)	Define la interfaz de alto nivel que usa el cliente. Guarda una referencia al implementador y le delega el trabajo de bajo nivel.
Abstracción Refinada (RefinedAbstraction)	Extiende la abstracción con variantes de la lógica de alto nivel.
Implementador (Implementor, interfaz)	Declara las operaciones de bajo nivel. No tiene por qué coincidir con las de la abstracción.
Implementador Concreto (ConcreteImplementor)	Implementa la interfaz del implementador con una variante específica.
Cliente	Crea el implementador concreto, se lo pasa a la abstracción por constructor y usa la abstracción.

Ventajas

- Desacopla la abstracción de la implementación: ambas pueden evolucionar por separado.
- Evita la explosión de clases: $M + N$ en lugar de $M \times N$.
- Principio Abierto/Cerrado: se agregan abstracciones o implementaciones nuevas sin modificar las existentes.
- Principio de Responsabilidad Única: la lógica de alto nivel queda separada de los detalles de bajo nivel.
- Permite cambiar la implementación en tiempo de ejecución.

- El cliente trabaja con la abstracción y no conoce los detalles de implementación.

Desventajas

- Aumenta la cantidad de clases e interfaces, y con ellas la complejidad del diseño.
- Para casos con una sola dimensión de variación es una sobreingeniería innecesaria.
- Exige identificar bien las dos dimensiones de cambio desde el diseño inicial.
- Agrega una capa de indirección, y el flujo es más difícil de seguir al leer el código.

Ejemplos de Aplicación

Ejemplo 1 - Pagos de un ERP (se programa)

Consigna

El área de tesorería de un ERP registra pagos. Hay dos tipos de pago: a proveedores y de sueldos. Cada pago se realiza por un medio, y cada medio tiene sus propias reglas:

- Transferencia bancaria: comisión del 0,5 % con un mínimo de \$5, sin límite de importe.
- Efectivo: sin comisión y hasta \$5.000 por operación.

Si el medio autoriza el pago, se genera el asiento contable (Debe: Proveedores o Sueldos a pagar, más Gastos bancarios si hay comisión; Haber: Banco o Caja). Si lo rechaza, no se genera ningún asiento. El usuario trabaja con un menú: 1) pago a proveedor, 2) pago de sueldo, 0) salir. Al registrar un pago, ingresa por teclado el nombre del beneficiario y el importe, y elige el medio. Se valida que el nombre no esté vacío, que el importe sea numérico y mayor a cero, y que las opciones del menú sean válidas.

Justificación

Hay dos dimensiones que varían de forma independiente: el tipo de pago (proveedor o sueldo) y el medio utilizado (transferencia o efectivo). Con herencia habría que crear PagoProveedorTransferencia, PagoProveedorEfectivo, PagoSueldoTransferencia y PagoSueldoEfectivo, y cada medio nuevo (por ejemplo, tarjeta) obligaría a crear dos clases más, una por cada tipo de pago. Con Bridge, un medio nuevo es una sola clase ($M + N$ en lugar de $M \times N$). Además:

- El pago (abstracción) define el título y la cuenta contable que corresponden a su tipo, y delega en IMedioPago la comisión y la autorización.
- Cada medio encapsula sus propias reglas, y el pago no sabe con cuál está trabajando.

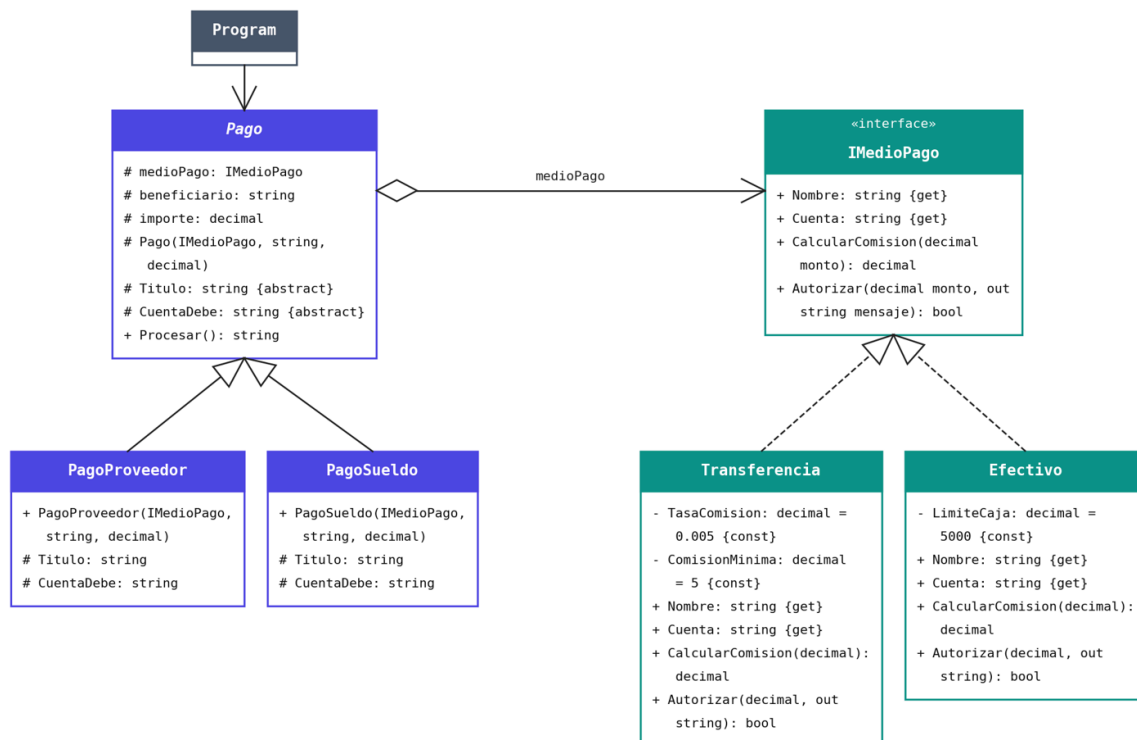
- Agregar TarjetaCredito es crear una clase sin tocar los pagos; agregar un pago de impuestos no obliga a tocar ningún medio.

Participantes

- **Abstracción:** Pago.
- **Abstracciones Refinadas:** PagoProveedor, PagoSueldo.
- **Implementador:** IMedioPago.
- **Implementadores Concretos:** Transferencia, Efectivo.
- **Cliente:** Program.

Comparación con el DC General

DC General	Ejemplo 1
Client	Program
Abstraction	Pago
RefinedAbstraction	PagoProveedor, PagoSueldo
Implementor	IMedioPago
ConcretelImplementorA, B	Transferencia, Efectivo
implementor	medioPago
operation()	Procesar()
operationImpl()	CalcularComision() y Autorizar()



Ejemplo 2 - Presupuestos y pedidos de venta de un ERP (se programa)

Consigna

El módulo de ventas de un ERP emite dos documentos: el presupuesto y el pedido de venta. El precio depende de la lista del cliente:

- Lista minorista: precio de lista, sin descuento.
- Lista mayorista: 15 % de descuento desde 20 unidades.

Cada documento calcula el importe bruto, el descuento, el neto, el IVA (21 %) y el total. Además, el presupuesto indica su validez (15 días) y que no reserva stock, y el pedido de venta indica el anticipo requerido (30 % del total) y el saldo a abonar en la entrega. El usuario trabaja con un menú: 1) crear presupuesto, 2) crear pedido de venta, 0) salir. Ingresar por teclado el cliente, el producto, el precio unitario y la cantidad, y elige la lista de precios. Se valida que los textos no estén vacíos, que el precio sea numérico y mayor a cero, que la cantidad sea un entero mayor a cero y que las opciones del menú sean válidas.

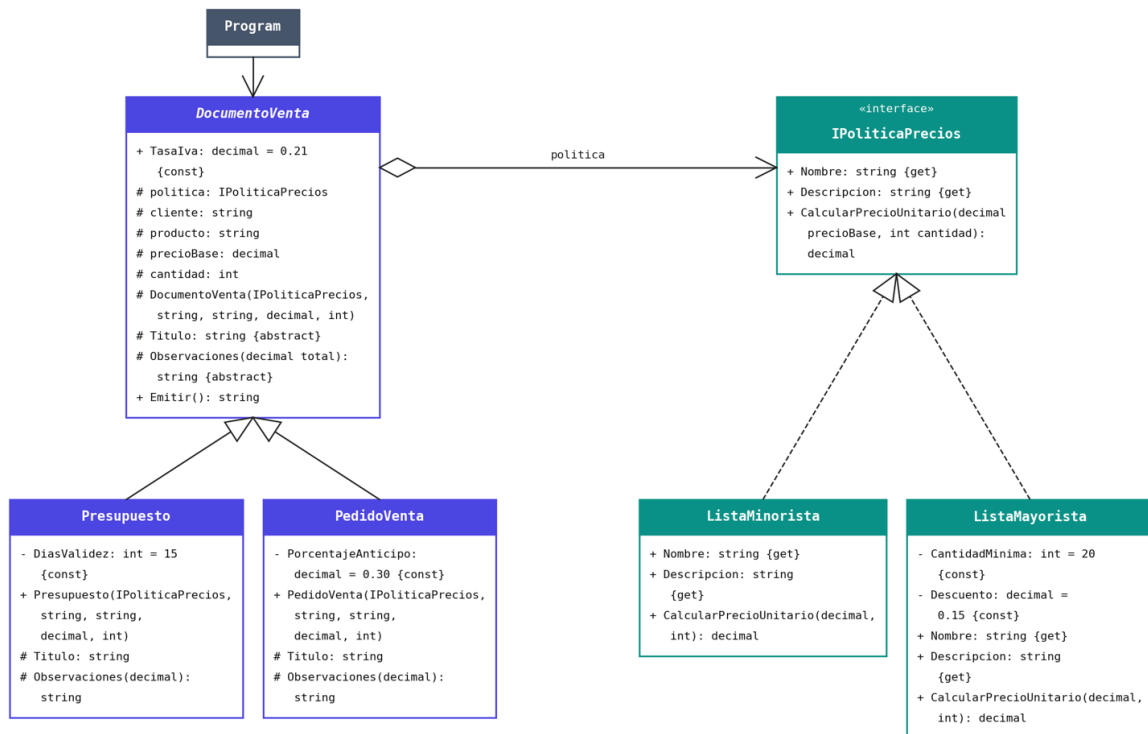
Justificación

Varían de forma independiente el tipo de documento (presupuesto o pedido) y la lista de precios (minorista o mayorista). Con herencia habría que crear PresupuestoMinorista, PresupuestoMayorista, PedidoMinorista y PedidoMayorista, y cada lista nueva (por ejemplo, corporativa) obligaría a crear dos clases más. Con Bridge, una lista nueva es una sola clase. Además:

- El documento (abstracción) calcula los importes y agrega sus observaciones propias (validez o anticipo), y delega en IPoliticaPrecios el cálculo del precio unitario.
- Cada lista encapsula su regla de descuento, y el documento no sabe con cuál está trabajando.
- Agregar ListaCorporativa no toca los documentos; agregar un remito no toca ninguna lista.

Participantes

- **Abstracción:** DocumentoVenta.
- **Abstracciones Refinadas:** Presupuesto, PedidoVenta.
- **Implementador:** IPoliticaPrecios.
- **Implementadores Concretos:** ListaMinorista, ListaMayorista.
- **Cliente:** Program.



Ejemplo 3 - Movimientos de stock con distintos métodos de valuación (no se programa)

Consigna

El módulo de inventario de un ERP controla el stock de un producto. Registra dos tipos de movimiento:

- **Ingreso por compra:** entra una cantidad a un costo unitario.
- **Egreso por venta:** sale una cantidad. Si no hay stock suficiente, el egreso se rechaza.

El costo de cada egreso depende del método de valuación que use la empresa:

- **PEPS (primero en entrar, primero en salir):** consume primero los lotes más antiguos, cada uno a su costo.
- **Precio promedio ponderado:** valúa la salida al costo promedio de todo el stock disponible.

Cada movimiento aprobado genera su asiento:

- Ingreso: Debe Mercaderías, Haber Proveedores.
- Egreso: Debe Costo de mercaderías vendidas, Haber Mercaderías.

Por ejemplo, si ingresan 10 unidades a \$100 y luego 10 a \$120, y se venden 15:

- Con PEPS, el costo es $10 \times 100 + 5 \times 120 = \1.600 .
- Con promedio ponderado, es $15 \times 110 = \$1.650$.

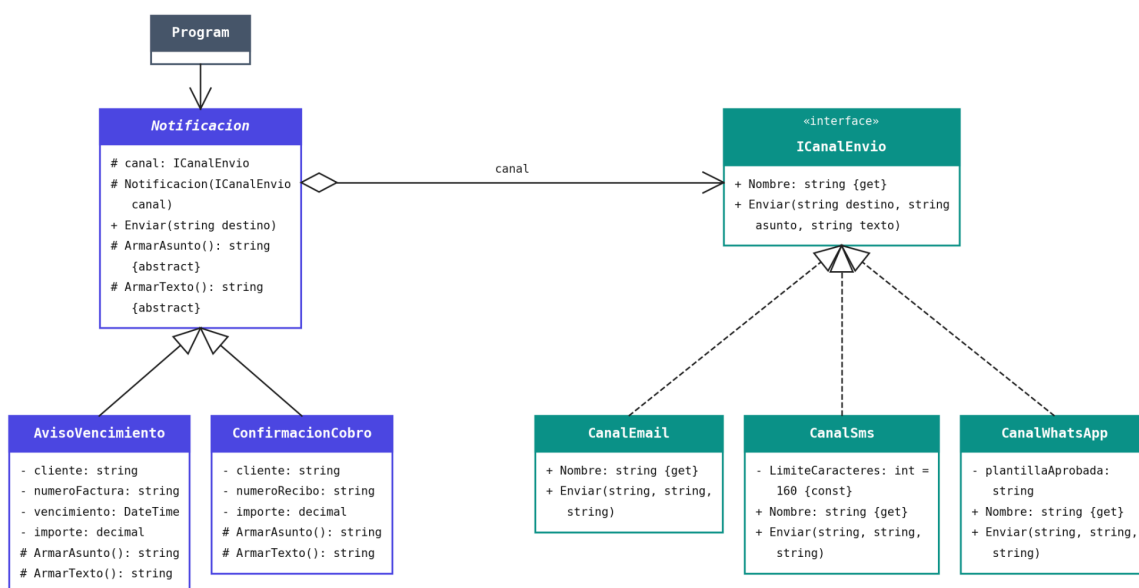
Justificación

Varían de forma independiente el tipo de movimiento (ingreso o egreso) y el método de valuación (PEPS o promedio). Con herencia habría que crear IngresoPeps, IngresoPromedio, EgresoPeps y EgresoPromedio, y cada método nuevo obligaría a crear dos clases más, una por cada tipo de movimiento. Con Bridge, un método nuevo es una sola clase. Además:

- El movimiento (abstracción) define sus reglas y su asiento, y delega en IMetodoValuacion el registro de lotes y el cálculo del costo.
- Cada método encapsula su algoritmo y su estado: PEPS guarda los lotes y el promedio guarda el costo promedio actual.
- El movimiento no sabe con qué método trabaja, así que cambiar de PEPS a promedio no obliga a tocar los movimientos.
- Agregar un movimiento nuevo, por ejemplo un ajuste de inventario, no toca ningún método.

Participantes

- Abstracción: MovimientoStock.
- Abstracciones Refinadas: IngresoStock, EgresoStock.
- Implementador: IMetodoValuacion.
- Implementadores Concretos: ValuacionPeps, ValuacionPromedio.
- Cliente: Program.



Chain of Responsibility

Clasificación

Chain of Responsibility es un patrón de comportamiento del catálogo GoF. Este tipo de patrones resuelven cómo se reparten las responsabilidades entre los objetos y cómo se comunican.

Descripción (que hace)

Evita acoplar al emisor de una solicitud con su receptor, dándole a más de un objeto la posibilidad de atenderla. Los receptores se encadenan y la solicitud pasa de uno a otro hasta que alguno la procesa.

El emisor no envía la solicitud a un objeto concreto, sino al primer eslabón de la cadena. Cada manejador conoce solo a su sucesor y decide si procesa la solicitud o la pasa al siguiente. Así, el emisor no sabe quién termina atendiendo su pedido y cada manejador no conoce la cadena completa.

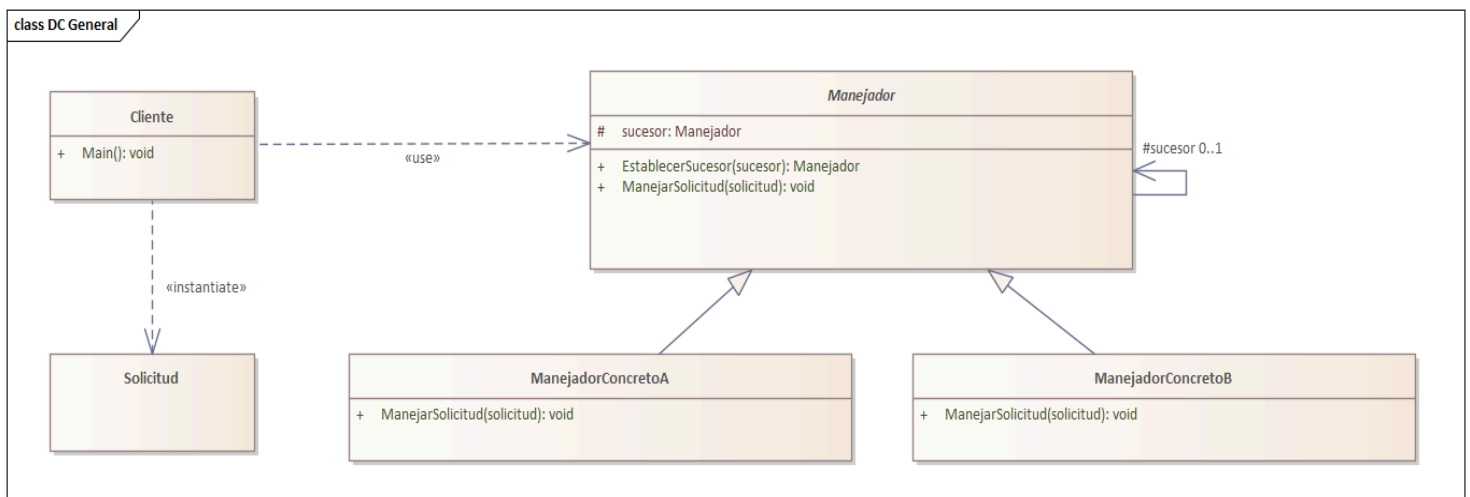
Problema que resuelve

- **Condicionales que concentran todas las reglas:** un if/else o switch decide quién atiende cada solicitud, y hay que modificarlo cada vez que cambia una regla o aparece un receptor nuevo.
- **Emisor acoplado a todos los receptores:** quien emite la solicitud tiene que conocer a cada posible receptor y saber cuál elegir.
- **Orden de procesamiento fijo:** cambiar el orden en que se evalúan los receptores obliga a reescribir el código que los recorre.

Cuando se aplica

- Cuando más de un objeto puede atender una solicitud y *el que la atiende no se conoce de antemano*, sino que depende de la propia solicitud.
- Cuando se quiere emitir una solicitud *sin especificar explícitamente el receptor*.
- Cuando el conjunto de receptores, o *el orden en que se los consulta*, debe poder definirse o cambiarse en tiempo de ejecución.
- Cuando una solicitud debe pasar por *una serie de pasos o validaciones* que pueden cortar el procesamiento.

Diagrama de Clases General



Participantes

Participante	Responsabilidad
Solicitud	Objeto con los datos del pedido que recorre la cadena.
Manejador (clase abstracta o interfaz)	Declara la operación para atender solicitudes, por ejemplo ManejarSolicitud(). Guarda la referencia al sucesor e implementa el reenvío por defecto.
Manejador Concreto	Atiende las solicitudes de las que es responsable. Si no puede, la pasa a su sucesor.
Cliente	Arma la cadena y envía la solicitud al primer manejador, sin saber cuál la va a atender.

Ventajas

- Reduce el acoplamiento: el emisor sólo conoce la interfaz del manejador, no a los receptores concretos.
- Principio Abierto/Cerrado: se agregan manejadores nuevos sin modificar al cliente ni a los manejadores existentes.
- Principio de Responsabilidad Única: cada manejador encapsula un único criterio.
- El orden y la composición de la cadena se pueden cambiar en el tiempo de ejecución.

Desventajas

- La recepción no está garantizada: si ningún manejador atiende la solicitud, sale de la cadena sin procesarse. Hay que definir qué pasa en ese caso.
- Es más difícil de seguir y depurar, porque no hay un receptor explícito.
- Una cadena larga puede afectar el rendimiento, ya que la solicitud puede recorrer muchos eslabones antes de ser atendida.

Ejemplos de Aplicación

Ejemplo 1 - Pago con medios alternativos (se programa)

Consigna

Una aplicación de pagos permite abonar compras con tres medios: el saldo de la billetera virtual, una tarjeta de débito y una tarjeta de crédito. Al pagar, el sistema intenta en ese orden:

- **Billetera:** cobra el pago si el saldo alcanza.
- **Tarjeta de débito:** si la billetera no alcanza, cobra si el saldo de la cuenta asociada alcanza.
- **Tarjeta de crédito:** si tampoco alcanza, cobra si el monto no supera el límite disponible.

El primer medio que puede cubrir el monto lo cobra y descuenta su saldo o límite. Si ninguno puede, el pago se rechaza. Al iniciar, el usuario carga por teclado el saldo de la billetera, el de la cuenta y el límite de la tarjeta; después ingresa los montos a pagar y el sistema informa con qué medio se cobró cada uno. Se valida que los valores ingresados sean numéricos: los saldos y el límite pueden ser cero, y los montos a pagar tienen que ser mayores a cero.

Justificación

Sin el patrón, Program necesitaría un if/else que consulte a cada medio en orden, y habría que modificarlo para agregar uno nuevo o cambiar la prioridad. Con Chain of Responsibility:

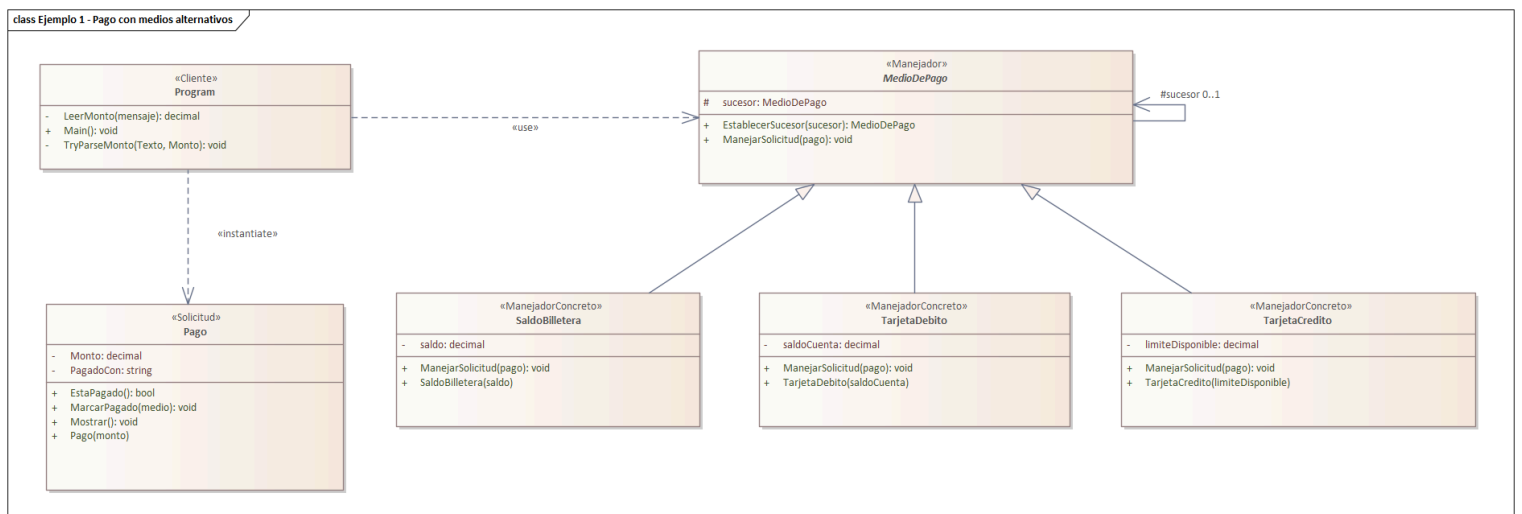
- cada medio decide por sí mismo si puede cubrir el pago, según su propio saldo o límite;
- Program solo conoce al primer medio, y la prioridad se define al armar la cadena.

A diferencia de una jerarquía de aprobación, acá cada eslabón tiene un estado que cambia con cada operación: dos pagos iguales seguidos pueden cobrarse con

medios distintos porque el primero consumió el saldo de la billetera. Agregar un medio nuevo, por ejemplo puntos de un programa de fidelidad, es sumar un eslabón. El pago rechazado muestra la principal desventaja del patrón: la solicitud recorre toda la cadena y nadie la atiende.

Participantes

- **Solicitud:** Pago.
- **Manejador:** MedioDePago.
- **Manejadores Concretos:** SaldoBilletera, TarjetaDebito, TarjetaCredito.
- **Cliente:** Program.



Comparación con el DC General

Este ejemplo sigue la estructura general sin agregados, para que la correspondencia se vea directamente. Las operaciones del patrón conservan el mismo nombre; solo cambian las clases:

DC General	Ejemplo 1
Cliente	Program
Manejador	MedioDePago
ManejadorConcretoA, ManejadorConcretoB	SaldoBilletera, TarjetaDebito, TarjetaCredito
Solicitud	Pago
sucesor, EstablecerSucesor(), ManejarSolicitud()	Sin cambios

Lo que agrega el ejemplo es el criterio de cada medio, dentro de su ManejarSolicitud(), y el saldo o límite de cada uno. La asociación del Manejador consigo mismo no dice cuántos eslabones hay ni en qué orden: eso lo decide Program al crear y enlazar los objetos.

```
MedioDePago billetera = new SaldoBilletera(saldo);
billetera.EstablecerSucesor(new TarjetaDebito(saldoCuenta))
.EstablecerSucesor(new TarjetaCredito(limite));
```

Ejemplo 2 - Moderación de mensajes de un foro (se programa)

Consigna

Un foro necesita moderar automáticamente los mensajes antes de publicarlos. Cada mensaje pasa, en este orden, por los siguientes filtros:

- **Spam:** si el mensaje tiene más enlaces que el máximo permitido, se bloquea y no se publica.
- **Lenguaje ofensivo:** reemplaza cada palabra prohibida por asteriscos y deja seguir el mensaje.
- **Datos personales:** oculta números de teléfono y direcciones de correo y deja seguir el mensaje.

Si el mensaje llega al final de la cadena, se publica con las modificaciones aplicadas. El usuario define por teclado el máximo de enlaces y las palabras prohibidas, publica mensajes (autor y texto) y puede ver los publicados. Para cada mensaje se muestra qué filtro lo modificó o lo bloqueó. Se valida que el máximo de enlaces sea un número entero mayor o igual a cero, que el autor y el texto no estén vacíos y que no se repitan palabras prohibidas.

Justificación

Sin el patrón, el método de publicación aplicaría los tres controles uno detrás de otro, mezclando reglas que no tienen relación entre sí. Este ejemplo aprovecha el patrón de una forma distinta al Ejemplo 1:

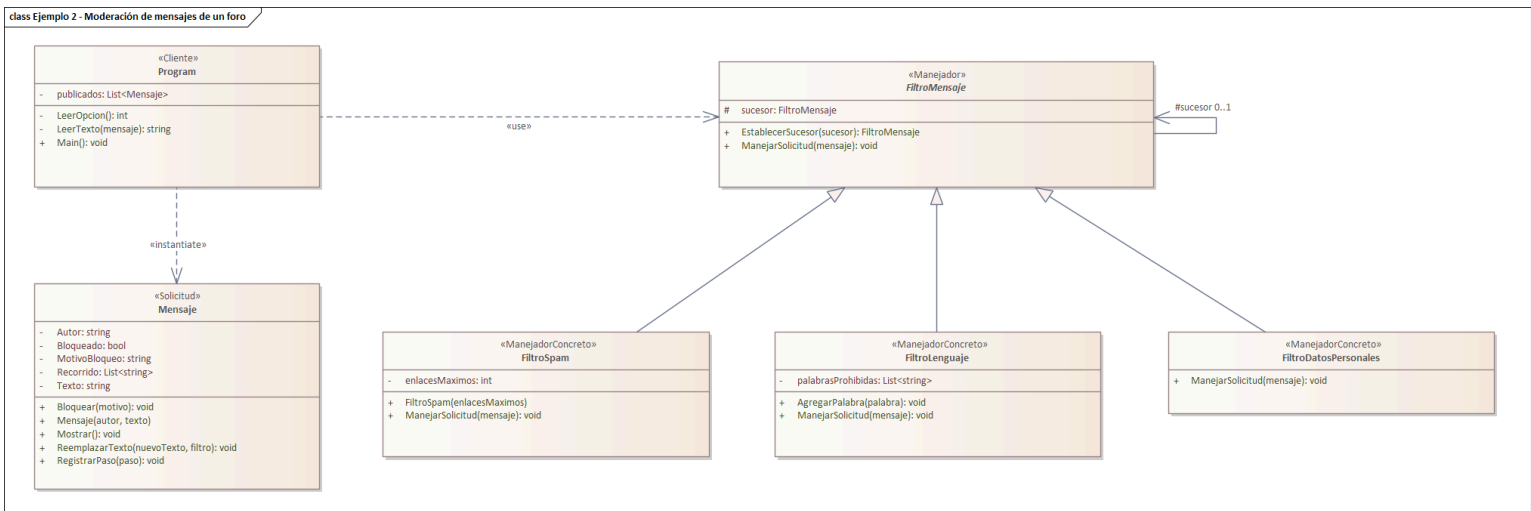
- Cada eslabón no solo decide: **procesa** la solicitud. Puede cortar la cadena (spam) o modificar el mensaje y pasarlo al siguiente (lenguaje y datos personales).
- Agregar un control nuevo, por ejemplo uno que limite la cantidad de mayúsculas, es una sola clase que se suma a la cadena, sin tocar los demás filtros ni a Program.

- El orden importa: el filtro de spam va primero porque no tiene sentido corregir un mensaje que después se va a bloquear.

En este caso llegar al final de la cadena no significa que nadie atendió la solicitud, sino que el mensaje superó todos los filtros y puede publicarse.

Participantes

- **Solicitud:** Mensaje.
- **Manejador:** FiltroMensaje.
- **Manejadores Concretos:** FiltroSpam, FiltroLenguaje, FiltroDatosPersonales.
- **Cliente:** Program.



Ejemplo 3 - Dispensador de billetes de un cajero (no se programa)

Consigna

Un cajero automático tiene billetes de \$20.000, \$10.000, \$2.000 y \$1.000, con una cantidad limitada de cada uno. Ante un pedido de extracción debe entregar el monto usando primero los billetes de mayor denominación disponibles. Si el monto no puede formarse exactamente, porque no es múltiplo de \$1.000 o porque faltan billetes, la operación se cancela y no se entrega nada.

Justificación

Cada dispensador entrega los billetes que puede de su denominación y pasa el monto pendiente a su sucesor. Es una tercera variante del patrón: en el Ejemplo 1 atiende uno solo, en el Ejemplo 2 todos procesan y alguno puede cortar, y acá varios manejadores aportan una parte de la misma solicitud.

- La cadena se ordena de mayor a menor denominación.
- Si al final queda un monto pendiente, el Cajero llama a Revertir() sobre la misma cadena para que cada dispensador recupere los billetes que había reservado. La cadena sirve tanto para procesar como para deshacer.

Todos los eslabones son instancias de la misma clase concreta con distinta denominación: la cadena es de objetos, no necesariamente de clases distintas. Para sumar un billete nuevo, por ejemplo de \$50.000, alcanza con agregar un eslabón al principio de la cadena.

Participantes

- **Solicitud:** SolicitudRetiro.
- **Manejador:** Dispensador.
- **Manejador Concreto:** DispensadorBilletes.
- **Cliente:** Cajero.

class Ejemplo 3 - Dispensador de billetes de un cajero

